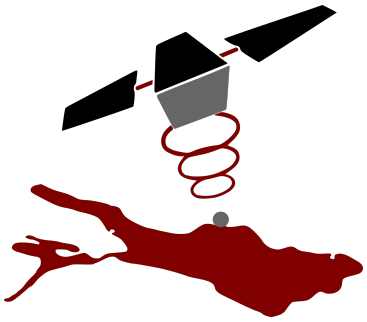
Verification- and Validation Plan

for

Development of a telemetry data link for the ERWIN cubesat



DHBW
Duale Hochschule
Baden-Württemberg
Ravensburg

Project:	HW/SW Projekt TSL - 0004
Date:	June 15, 2026
Version:	1.0
Description:	The Verification- and Validation Plan for the implemented TM chain

Revision History

Name	Date	Description	Version
Robin Eilers, Matthias Fuchs, Hannah Lindner, Lukas Reil, Nico Tunkowski	24.05.2026	Initial draft	1.0

Contents

1 Introduction 1
1.1 Purpose and Scope 1
1.2 Responsibility and Change Authority 1
1.3 Definitions 1

2 Applicable and Reference Documents 1
2.1 Applicable Documents 1
2.2 Reference Documents 1
2.3 Order of Precendence 2

3 System Description 2
3.1 System Architecture 2
3.2 End Item Architectures 3
3.3 CCSDS 132 Encoder 3
3.4 CCSDS 131 Encoder 3
3.5 CCSDS 131 Decoder 4

3.6 CCSDS 132 Decoder . . . 6
3.7 Test Equipment . . . 7
3.7.1 External Interface . . . 7
3.7.2 Data Generator . . . 8
3.7.3 Data Comparator . . . 8

4 Verification and Validation Process . . . 8
4.1 Verification Methods . . . 8
4.1.1 Analysis . . . 8
4.1.2 Inspection . . . 9
4.1.3 Test . . . 9
4.2 Validation Methods . . . 9
4.2.1 Inspection . . . 9
4.2.2 Demonstration . . . 9

5 End Item Verification and Validation . . . 10
5.1 CCSDS 132 Encoder / Decoder . . . 10
5.1.1 Engineering Unit Evaluations . . . 10
5.1.2 Verification Activities . . . 10
5.2 CCSDS 131 Encoder / Decoder . . . 11
5.2.1 Engineering Unit Evaluations . . . 12
5.2.2 Verification Activities . . . 12
5.3 Validation Activities . . . 14
5.3.1 Verification by Testing . . . 14

6 System Verification and Validation . . . 14
6.1 System Validation: Test Group 1 . . . 15
6.1.1 Test Definition: Test Group 1 . . . 15
6.1.2 Test Result . . . 16
6.2 System Validation: Test Group 2 . . . 17
6.2.1 Test Definition: Test Group 2 . . . 17
6.2.2 Test Result . . . 18

1 Introduction

1.1 Purpose and Scope

This document describes the testing procedures required to verify and validate the TM Chain developed for SeeSat e.V. within in the Software-Hardware Projekt course. The purpose of the V&V Plan is to identify the activities that will establish compliance with the requirements (verification) and to establish that the system will meet the customers' expectations (validation).

1.2 Responsibility and Change Authority

This plan will be maintained by the entire working group, with changes made as they see fit. Changes to the plan after version 1.0 may be made by any individual person of the team, but only after discussing them with the other team members.

1.3 Definitions

2 Applicable and Reference Documents

2.1 Applicable Documents

2.2 Reference Documents

- CCSDS-131.0-B-5
- CCSDS-132.0-B-3
- AMBA AXI-Stream Protocol Specification
- Development Process - HW/SW Projekt TSL - 0004

2.3 Order of Precendence

3 System Description

This chapter describes the system that this plan covers. Its purpose is to give proper context for anybody reading this plan without requiring any further documents. Knowledge of the relevant CCSDS standards is highly beneficial for the reader's understanding of the system.

3.1 System Architecture

The system under test, as pictured in figure 3.1 aims to implement the TM Space Data Link Protocol layer of CCSDS-132 and the TM Synchronization and Channel Coding layer of CCSDS-131. It is divided into 4 major subsystems, namely one encoder and one decoder for each of the relevant layers. The OnBoard-Computer (OBC) and Ground Support Equipment (GSE) are not within the scope of this system, but may be briefly mentioned, as their interfaces are shared with parts of the system. The communication between the the subsystems is implemented using the AMBA AXI-Stream protocol.

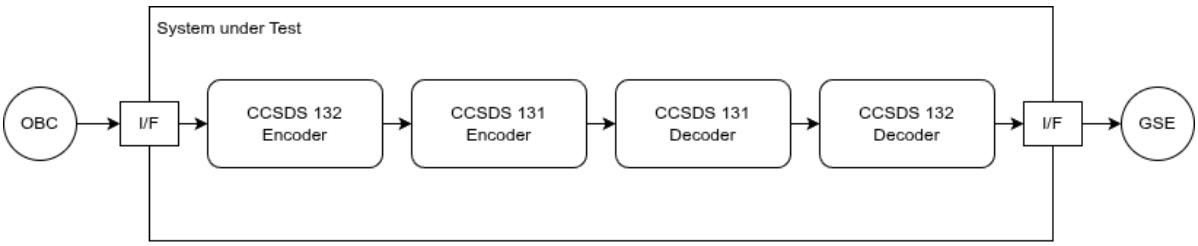


Figure 3.1: System Architecture

3.2 End Item Architectures

3.3 CCSDS 132 Encoder

3.4 CCSDS 131 Encoder

In general, the CCSDS 131 Encoder can be subdivided into four components defined by CCSDS 131.0-B-5. The first of these is the Reed-Solomon encoder, which is designed to implement a (223, 255)-RS encoder. This means that upto 16-byte errors can be corrected by the code generated here. The R/S encoder reads a one-byte-wide data stream from higher layers and sends a one-byte-wide data stream to lower layers of the CCSDS 131.0-B-5.

The following components, namely the RNG, ASM and CC, require a one-bit-wide datastream. This is why an auxiliary module, the width converter, is introduced. This converts a 8-bit-wide datastream to a one-bit-wide datastream. This module allows a clear separation of concern between the Reed-Solomon encoder and the RNG unit. Neither element requires knowledge of the width of the data stream used in the other layers. This allows for low coupling between these layers and enables components to be changed if required.

The output stream can then be sent to the pseudorandomiser (RNG) module, which implements a linear feedback shift register using a generator polynomial to generate a random sequence. This randomisation is reset with each R/S code word, thus generating the same random sequence for each code word. The random bit value is then added to the one-bit stream received from the R/S encoder and sent to the ASM unit.

The Attached Sync Marker (ASM) module prepends a synchronisation pattern to the data stream generated by the RNG module every R/S code word, allowing the R/S decoder and RNG unit on the receiving side to resynchronise. It then forwards all data generated by the RNG module to the convolutional code (CC) module, along with the added synchronisation patterns.

The CC module implements a convolution over the received datastream from the ASM unit. This convolution is implemented over two dif-

ferent polynomials, thus generating a datastream with a data rate twice that of the input data rate while continuously changing between the two polynomials. The output of this module is the final stage of the 131 encoder and will be sent to the physical layer.

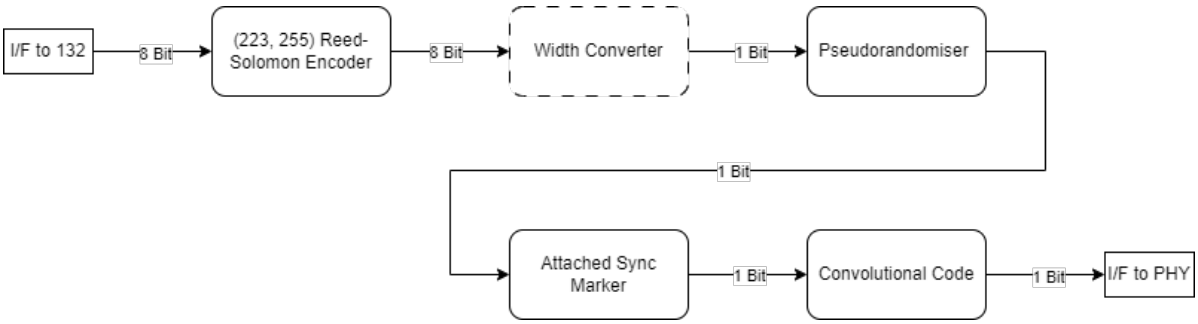


Figure 3.2: Internal Architecture of the 131 Encoder

3.5 CCSDS 131 Decoder

Similar to the CCSDS 131 encoder, the decoder can also be divided into four components, as defined by CCSDS 131.0-B-5. The first element is the convolutional decoder. The general goal of this decoder is to recompute the most likely pattern of bits inputted into the CC. In this implementation, this is achieved using a COTS Viterbi decoder. The most probable sequence of bits is then output to the ASM detection unit.

This unit attempts to match the ASM pattern generated by the ASM module on the encoder side with the bitstream generated by the convolutional decoder. This is achieved by calculating the Hamming distance to the expected pattern and triggering an ASM detection if the Hamming distance is lower than a predefined value. Additionally, a counter is implemented to allow for a minimum distance between two detected ASMs, to prevent false detections.

As with the encoder element, the RNG unit generates a randomisation sequence and adds it to the incoming bit stream from the ASM decoder. If an ASM is successfully detected, the RNG unit is reset again to synchronise it with the Reed-Solomon code word. The goal of this unit is to restore the original sequence generated by the Reed-Solomon encoder (including errors not fixed by the Viterbi decoder).

Another auxiliary unit is a second width converter, which converts a one-bit data stream into an eight-bit data stream, thus preparing the data stream for decoding by the R/S decoder.

In contrast to the Viterbi Decoder, the R/S Decoder Unit attempts to find a hard algebraic solution to errors that may have been introduced by the transmission link and not fixed by the Viterbi Decoder. From an architectural point of view, four sub-components are required for this decoder. The first element calculates the syndromes (i.e. the 'illness' of the code), and this information is then forwarded to the next sub-components. This solves the system of equations defined by the syndromes, and finds error locators and error magnitude. These polynomials can then be evaluated by the third sub-component to find the exact error positions and magnitude of the errors. These can then be added to the data stream, which is delayed by the last sub-component. This defines a long FIFO that delays the full data stream for the computational time required by the other three sub-components. The R/S decoder then outputs the corrected datastream and a Reed-Solomon failure flag if decoding failure is detected.

It should be noted that the entire CCSDS 131.0-B-5 decoder outputs a corrupted data stream if the data cannot be corrected; this ensures synchronisation with higher layers. The R/S Decider is therefore the final element of the CCSDS 131.0-B-5 chain, thereby defining the boundary of this subsystem's system.

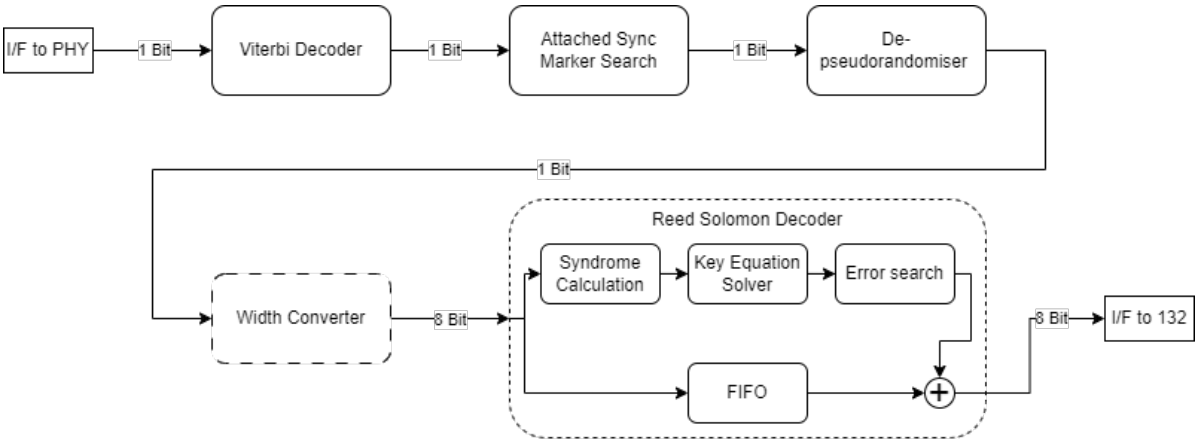


Figure 3.3: Internal Architecture of the 131 Decoder

3.6 CCSDS 132 Decoder

The CCSDS 132 Decoder is the ground component of the CCSDS 132-0-b3 Data Link Protocol Sublayer. The decoder is mainly responsible for the extraction of the Space Packets transmitted from on board the satellite. The other data units are not yet implemented and not covered in this document. In order to transmit the Space Packets, the Encoder chain encapsulates them into a TM Transfer frame, from now on called frame. The encoding is described in chapter 3.3.

The decoding chain is composed of four stages to demultiplex the different master and virtual channels, indentified by its master channel and virtual channel id, from each other. Figure 3.4 shows the data flow of the space packet data from ints source at the 131-decoder to its destination, the AXI 4 Stream FIFO. The complete data flows is positioned in the programmable logic of the SoC with the FIFO as the interface to the processing system. This interface represents the only yet defined service, the packet service.

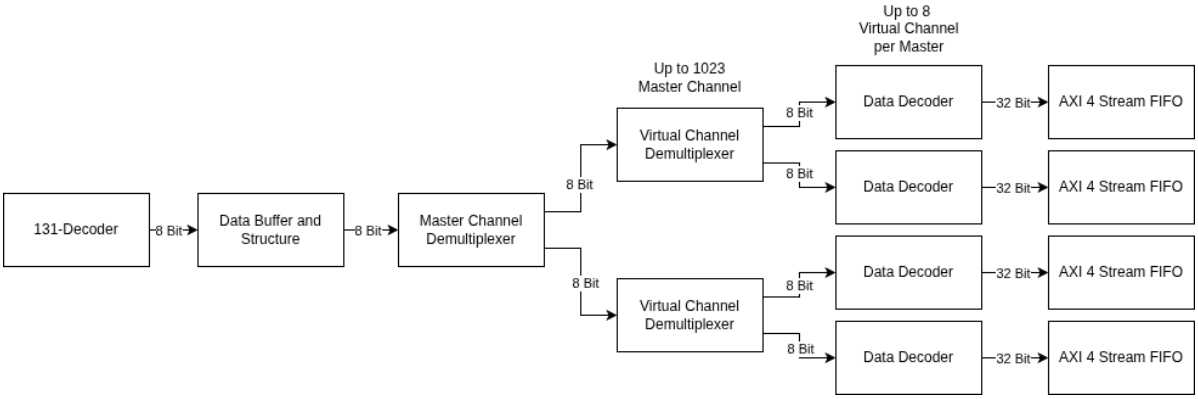


Figure 3.4: Internal Architecture of 132 Decoder

The Data Buffer and Structure entity recieves the frame from the 131 decoder in octets, because the frame itself is octet aligned. The purpose of this entity is to buffer the entire frame, decode the header and forward the data field to the master channel demultiplexer. In adition it will delete all only idle data frames.

The master channel demultiplexer routes the data accourding to the provided master channel id to a specified virtual channel demultiplexer.

There is one master channel demultiplexer per physical channel. The cahnnel organisation itself is specified by the user and therefore customizable.

The virtual channel demultiplexer routes the data to its designated virtual channel identified by the virtual channel id. There is one instanstance per master channel.

The data decoder recieves the data octets per virtual channel and buffers them once again to decode the space packet header and to check wether the packet is complete. Idle packets and incomplete packets are discarded. In addition to the data, the first header pointer is provided to recieve packets that span over multiple frames. If there is a missmatch between remaining packet data and first header pointer, the partly receive packet will be discarded as well and the first header pointer defines the start of the next valid packet header. The output interface is a 32Bit AXI 4 stream interface. Because of the 8Bit to 32Bit conversion, it is possible, that the output 32Bit will not be filled completely and up to 3 octets will remain in the data decoder until new data is provided at the sending end.

3.7 Test Equipment

Test Equipment is equipment not required by any of the system's requirements, but rather necessary for any of the tests described in this plan. It consists of three important components: the external interface, the Data Generator and the Data Comparator.

3.7.1 External Interface

The external interface connects the other components, which are implemented in the programmable logic part of the SoC to the ethernet interface implemented in the processing system. Through this external interface the test data may be injected into the Data Generator and the output of the Data Comparator is being reported to a connected computer. For interfacing between PL and PS the AXI-4 (lite) protocol is used.

3.7.2 Data Generator

The Data Generator allows the Testsystem to input data into the TM chain. Is uses the CCSDS 132 Encoder's native interface and simulates a connected On-Board Computer by providing valid Space Packets to be sent through the TM chain.

3.7.3 Data Comparator

The Data Comparator receives the data output by the CCSDS 132 Decoder and compares it with the delayed output of the Data Generator. If the output of the decoder matches the expected output, this is reported to the external interface. If they don't match, a bitwise difference is reported for debugging purposes.

4 Verification and Validation Process

4.1 Verification Methods

Verification methods are all methods to proof a implementation adheres to its given requirements.

4.1.1 Analysis

An analysis is conducted using a mathematical representation / model of the implemented system. Certain requirements are not easily testable and not inspectable. The requirement is verified by using this model to perform a rigorous mathematical proof.

4.1.2 Inspection

An inspection is done in groups not smaller then two people, where at least one person who has not contributed to the unit under inspection is present.

An inspection's purpose is to verify the correct implementation of a given requirement, which can not be tested using automated tests, as per chapter 4.1.3.

4.1.3 Test

A test verifying a requirement must be implemented by a person, who has not contributed to the unit under test. This rule may be relaxed for integration tests, where multiple people have contributed to the system under test. In that case no test shall be executed without verification by inspection by at least on other person.

A test of VHDL code shall be performed using Vivado's builtin simulation suite. A test shall use assert statements to automatically falsify an executed test, if the tested requirement is not fullfilled.

4.2 Validation Methods

4.2.1 Inspection

An inspection may also be performed to validate a certain property of the system, e.g. the internal structure or implementation details.

4.2.2 Demonstration

A demonstration is conducted either using a simulation or using an implementation of the developed system under test, which is loaded into the target FPGA. It does not specifically verify any requirement, but serves to show a customer the correct high level functionality requested.

5 End Item Verification and Validation

The white box tests are designed to demonstrate the compliance with the CCSDS' requirements and aid in the continuous integration of the product.

5.1 CCSDS 132 Encoder / Decoder

The development of the CCSDS 132 Encoder and Decoder end items is closely coupled. Therefore, all verfication and validation activities are done together for both subsystems. This not only simplifies planning but also allows the employment of better verification methods.

5.1.1 Engineering Unit Evaluations

No strict guidelines are given for the development of the end items. As per the set development process every developed unit shall have appropriate unit tests to verify correct functionality.

5.1.2 Verification Activities

The verification activities aim to cover 100 % of the system's high level requirements, either by defining test cases to be tested in a testing campaign or, where tests are not applicable / possible, by inspection in a code walkthrough.

5.1.2.1 Verification by Testing

Too allow for a more flexible test process on the board the tests where grouped into two categories, the first one is the test that only requires a AXI-Interface for writing and reading from the system, this group of tests shall be called *Test Group 1* in the following. The second test group from now on called *Test Group 2* requires multiple AXI-Interfaces to by

synthesized into the system since these test multiple Virtual channels and multiple master channels.

5.1.2.1.1 Test Group 1 Test Group 1 covers the requirements detailed in Table 5.1. These requirements focus on the general operation of the system, these include testing of interfaces and the complete TM Chain tests by sending data into the system and expecting that the exact data is returned. They also tests if the correct Hardware is used and the configuration of it is as specified.

Covered Requirement	Imp. Developer
GND-REQ-3	
GND-REQ-4	
GND-REQ-5	
GND-REQ-13	
GND-REQ-14	
SPC-REQ-2	
SPC-REQ-3	
SPC-REQ-6	
SYS-REQ-1	
SYS-REQ-2	

Table 5.1: Requirements covered by Test Group 1

5.1.2.1.2 Test Group 2 Test Group 2 covers the requirements detailed in Table 5.2. These tests define usage of multi master and multi virtual channel systems, there fore needing different software to stimulate the system accordingly. They also include some edge cases that require extra attention. This multi interface setup is quite similar to the decoder interfaces elaborated in Figure 3.4.

5.2 CCSDS 131 Encoder / Decoder

The development of the CCSDS 131 Encoder and Decoder end items is closely coupled. Therefore, all verfication and validation activities are done together for both subsystems. This not only simplifies planning but also allows the employment of better verification methods.

Covered Requirement	Imp. Developer
GND-REQ-15	
SPC-REQ-13	
SPC-REQ-16	
SPC-REQ-17	
SPC-REQ-18	

Table 5.2: Requirements covered by Test Group 2

5.2.1 Engineering Unit Evaluations

No strict guidelines are given for the development of the end items. As per the set development process every developed unit shall have appropriate unit tests to verify correct functionality.

5.2.2 Verification Activities

The verification activities aim to cover 100 % of the system's high level requirements, either by defining test cases to be tested in a testing campaign or, where tests are not applicable / possible, by inspection in a code walkthrough.

5.2.2.1 Verification by Testing

5.2.2.2 Verification by Inspection

5.2.2.2.1 Inspection RNG and ASM Git Hash 6816daf: 01.06.2026

Covered Requirement	Imp. Developer	Ver. Developer	PASS/FAIL
GND-REQ-9	HL	MF LR	PASS
GND-REQ-10	HL	MF LR	PASS
SPC-REQ-23	HL	MF LR	PASS
SPC-REQ-24	HL	MF LR	PASS
SYS-REQ-3	HL	MF LR	PASS
SYS-REQ-4	HL	MF LR	PASS
NFUNC-REQ-1	HL	MF LR	FAIL, AXI Signale
NFUNC-REQ-2	HL	MF LR	PASS
SPC-REQ-26	HL	MF LR	PASS

Table 5.3: Requirements covered by Inspection RNG and ASM.

5.2.2.2.2 Inspection Reed Solomon Git Hash 7ed3168: 01.06.2026

Covered Requirement	Imp. Developer	Ver. Developer	PASS/FAIL
GND-REQ-1	MF	HL LR	PASS
GND-REQ-11	MF	HL LR	PASS
GND-REQ-12	MF	HL LR	PASS
GND-REQ-22	MF	HL LR	PASS
SPS-REQ-22	MF	HL LR	PASS
SYS-REQ-5	MF	HL LR	PASS
SYS-REQ-6	MF	HL LR	PASS
SYS-REQ-9	MF	HL LR	PASS
NFUNC-REQ-1	MF	HL LR	FAIL
NFUNC-REQ-2	MF	HL LR	FAIL
SPC-REQ-26	MF	HL LR	PASS
GND-REQ-2	LR	MF HL	PASS

Table 5.4: Requirements covered by Reed Solomon

5.2.2.2.3 Inspection Convolutional Code Git Hash 6816daf: 01.06.2026

Covered Requirement	Imp. Developer	Ver. Developer	PASS/FAIL
GND-REQ-5	LR	MF HL	PASS
GND-REQ-7	LR	MF HL	PASS
GND-REQ-8	LR	MF HL	PASS
SPS-REQ-5	LR	MF HL	PASS
SPS-REQ-21	LR MF	HL	PASS
SPS-REQ-25	LR	MF HL	PASS
SYS-REQ-7	LR	MF HL	PASS
SYS-REQ-8	LR	MF HL	PASS
NFUNC-REQ-1	LR	MF HL	FAIL, AXI Signale, Reset
NFUNC-REQ-2	LR	MF HL	PASS
SPC-REQ-26	LR	MF HL	PASS
GND-REQ-2	LR	MF HL	PASS

Table 5.5: Requirements covered by Convolutional Code

5.2.2.3 Verification by Analysis

5.2.2.3.1 GND-REQ-14

5.2.2.3.2 SPC-REQ-1

5.2.2.3.3 SPC-REQ-2

5.2.2.3.4 SPC-REQ-4

5.2.2.3.5 SPC-REQ-6

5.2.2.3.6 SPC-REQ-8

5.3 Validation Activities

5.3.1 Verification by Testing

6 System Verification and Validation

The purpose of the black box test is not to verify the correct implementation of the CCSDS' requirements, but rather to validate that no data is lost and the desired data rate is achievable.

This requires the entire processing chain to be fully integrated, with a system in place to simulate the On-Board Computer (OBC), which handles data generation and comparison of the generated data with the output of the integrated chain. This Testsystem will be run on the target hardware to also demonstrate the correct synthesis and implementation of the devloped product. The described architecture is visualized in Figure 6.1

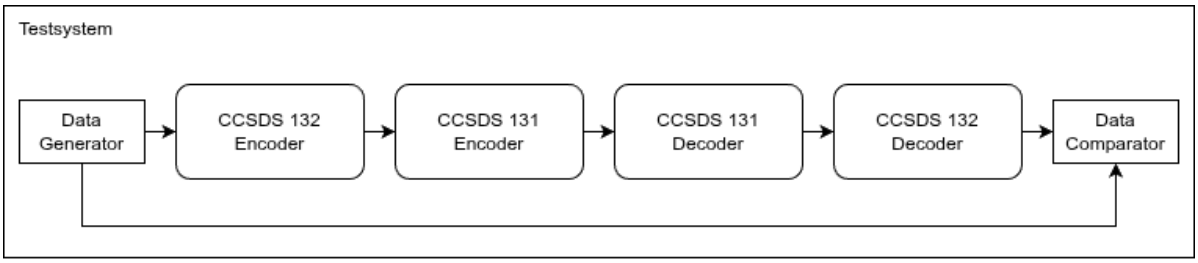


Figure 6.1: End to End Testsystem

6.1 System Validation: Test Group 1

6.1.1 Test Definition: Test Group 1

The overall system is validated by writing predefined data into the CCSDS 132 Encoder and observing the CCSDS 132 Decoder's output. If the data generated by the Data Generator matches the delayed output of the Decoder, the Data Comparator will indicate this to the user as success. If the data does not match, a failure is reported instead.

In general to test Test Group 1 the tester is required to use the following Software: AMD Vitis™ Unified Software Plattform, Python 3.x, Wireshark and Tasks Manager. This test amuses that the tester uses a Windows 11 based system. The test setup shall consist of the device under test, this should be a PYNQ Z2 board or equivalent. Adding to that a Host-PC shall be connected to the DUT using both a USB type A micro and an Ethernet cable. The Host-PCs IP-interface shall be configured using the parameters defined in Table 6.1.

Parameter	**Value**
IP-Address	192.168.1.1
Subnetmask	255.255.255.0
Standard gateway	N/A

Table 6.1: Parameters for the IP-interface of the Host-PC

The test software written for the embedded CPU shall be uploaded using the Vitis tool chain, it might be beneficial to use the build in debugger for some values. Additionally the python script for the Host-PC shall be launched to stimulate the DUT and to read the values back.

Req	Step	Expected Result	P/F
GND-REQ-5 SPS-REQ-6	Evaluate the Board used for testing and check if its a PYNQ Z2 board	Its a PYNQ Z2 board	
SPC-REQ-3	Start the test system on both the Host-PC and the embedded CPU	no errors or exceptions are printed and the system is shows the data generated and the that is data received	
GND-REQ-3 SPS-REQ-2	Open the program Wireshark, select the corresponding ethernet interface	there are TCP Packets send from and to the IP-Address 192.168.1.10	
SPC-REQ-2	Open Task Manager and check the network tab	the network usage of the IP-Interface used for testing $\geq$ 4 Mbit/s [1]	
GND-REQ-4 GND-REQ-13 SYS-REQ-1 SYS-REQ-2	Analyise the data send into the system and compare with the data recieved	These should have the same byte order and the same data	

Table 6.2: Test definition for Test Group 1

6.1.2 Test Result

Req	Note	P/F
GND-REQ-5 SPS-REQ-6		
SPC-REQ-3		
GND-REQ-3 SPS-REQ-2		
SPC-REQ-2		
GND-REQ-4 GND-REQ-13 SYS-REQ-1 SYS-REQ-2		

Table 6.3: Test results for Test Group 1

6.2 System Validation: Test Group 2

6.2.1 Test Definition: Test Group 2

To test different Virtual and master channels a quite similar test setup to the first test group may be used, but instad of using an Hardware Synthises that exposes only one Virtual channel one shall be used where multiple VC and MC are exposed. Addtionaly a new embedded software is requiered to stimulate multiple channels. These shall output data to different ports to the Host-PC. Thus also a modifiered version of the test system written in python is requiered.

The test system will then stimulate multiple VC and MCs and therefore allow for validation of these MCs VCs.

Req	**Step**	**Expected Result**	**P/F**
GND-REQ-15 SPC-REQ-16 SPC-REQ-17	Stimulate difrend Virtual channels using the test system	the data stimulated matches the data recieved with their VC IDs	
GND-REQ-15 SPC-REQ-18	Stimulate difrend master channels using the test system	the data stimulated matches the data recieved with their MC IDs	
SPC-REQ-13	Stimulate using space packets of length 7 Octet	Expect that this now sub dived space packets are recieved and repacked correctly	
SPC-REQ-13	Stimulate using space packets of length 53 Octet	Expect that this now sub dived space packets are recieved and repacked correctly	
SPC-REQ-13	Stimulate using space packets of length 223 Octet	Expect that this now sub dived space packets are recieved and repacked correctly	
SPC-REQ-13	Stimulate using space packets of length 251 Octet	Expect that this now sub dived space packets are recieved and repacked correctly	

Table 6.4: Test definition for Test Group 2

6.2.2 Test Result

Req	Note	P/F
GND-REQ-15 SPC-REQ-16 SPC-REQ-17		
GND-REQ-15 SPC-REQ-18		
GND-REQ-3 SPS-REQ-2		
SPC-REQ-13		
SPC-REQ-13		
SPC-REQ-13		
SPC-REQ-13		

Table 6.5: Test results for Test Group 2